1. Project Title

Hotel Reservation System — a web-based platform that allows guests to search room availability, book and manage reservations online, and allows hotel staff to manage rooms, bookings, and guest records through a centralized system.

2. Problem Statement

Many hotels still use manual booking methods like phone calls or paper records. This can cause:

- Double room bookings
- Slow booking process
- Difficulty checking room availability
- Customer information errors
- Poor customer service
- No central booking system

3. Project Objectives

1. Provide guests with an online interface to search room availability and make reservations.
2. Allow hotel staff/admin to manage room inventory, pricing, and booking status in real time.
3. Eliminate double-booking through centralized, synchronized availability data.
4. Maintain digital records of guests, bookings, and payments for faster retrieval.
5. Deliver a functional MVP within the semester timeline using an iterative, client-feedback-driven approach.
6. Build a modular codebase that can be extended in future labs (design patterns, testing, deployment).

4. Stakeholder Analysis

Stakeholder	Role / Interest
Guests	Search rooms, make reservations, view booking history, make payments.
Receptionist	Manage check-in/check-out, view daily bookings, update room status.
Hotel Manager / Admin	Oversee room inventory, pricing, staff accounts, and generate reports.
Hotel Owner (Client)	Sponsors the project; defines business requirements and success criteria.
Development Team	Designs, builds, tests, and delivers the system .

5. Project Scope

The Hotel Reservation System will provide online hotel booking services for customers and management tools for hotel staff.

Included in Scope

- User registration and login
- Room search and availability checking
- Online room reservation
- Payment management
- Customer profile management
- Admin & Receptionist dashboard
- Room management
- Reservation reports
- Booking History

Out of Scope

- Employee payroll system
- Inventory management
- Multi-hotel chain management
- AI recommendation system
- SMS notification

6. Functional Requirements

The system shall provide the following functionalities (minimum 10):

1. The system shall allow guests to register and log in using email and password.
2. The system shall allow guests to search available rooms by date, room type, and price range.

3. The system shall allow guests to book a room for selected check-in and check-out dates.
4. The system shall prevent double-booking by validating room availability before confirming a reservation.
5. The system shall allow guests to view, confirm and cancel their own bookings.
6. The system shall allow hotel receptionist/admin to log in to a separate management dashboard.
7. The system shall allow admin to add, update, or remove room records (type, price, capacity, status).
8. The system shall allow admin to view a list of all current and upcoming bookings.
9. The system shall allow admin to update booking/room status (e.g., Available, Booked, Checked-in, Checked-out).
10. The system shall generate a booking confirmation with a unique reservation ID for each successful booking.
11. The system shall allow admin to search and filter bookings by guest name, date, or room number.
12. The system shall maintain a guest profile with contact details and booking history.

7. Non-Functional Requirements

The system shall satisfy the following quality attributes :

1. Usability: The interface shall be simple and intuitive for non-technical hotel staff and guests.
2. Performance: Room availability search results shall load within 2 seconds under normal load.
3. Security: Passwords shall be stored using secure hashing; role-based access shall separate guest and admin views.
4. Reliability: The system shall correctly prevent conflicting bookings for the same room and date range.
5. Availability: The MVP shall be accessible without unexpected downtime during demonstration and evaluation.
6. Maintainability: The codebase shall be modular, organized into clear layers (UI, business logic, data).
7. Scalability: The database design shall support adding more rooms, hotels, or branches without redesign.
8. Portability: The application shall run consistently across common browsers/devices used for the demo.
9. Data Integrity: Booking and payment records shall remain consistent even under concurrent access.

8. User Stories / Use Cases

1. As a guest, I want to search for available rooms by date and type, so that I can find a room that fits my needs.
2. As a guest, I want to book a room online and receive a confirmation, so that I don't need to call the hotel.
3. As a guest, I want to view and cancel my existing bookings, so that I can manage my travel plans easily.
4. As a hotel admin, I want to view and update room availability, so that guests always see accurate information.
5. As a hotel admin, I want to see a dashboard of today's check-ins and check-outs, so that front-desk operations run smoothly.

9. Assumptions and Constraints

Assumptions

- The hotel has a fixed, known set of room types (e.g., Single, Double, Deluxe, Suite).
- Online payment integration is out of scope for the MVP; only booking status is tracked.
- Only one hotel branch is managed in this phase (multi-branch support is a future extension).

Constraints

- The MVP must be completed within the current lab cycle using team-available tools and skills.
- The team consists of 3 members with shared responsibility across frontend, backend, and documentation.
- No dedicated hosting budget — the MVP will run locally or on a free-tier hosting service for demonstration.

10. Selected Software Process Model

Selected Model: Rapid Application Development (RAD)

RAD is the model suggested for this project and is well aligned with its characteristics: a well-understood business domain (hotel booking), a UI-heavy application, a small dedicated team, and a fixed academic timeline that benefits from fast, iterative prototyping and frequent client (instructor) feedback.

11. Justification of Process Model

RAD is suitable for the Hotel Reservation System for the following reasons:

- Well-understood domain: Hotel booking is a familiar business process, so requirements can be captured quickly without extensive research, which is a prerequisite for RAD to work well.

- UI-centric application: The core value of the system (search, book, manage) is delivered through the interface, and RAD emphasizes rapid prototyping of user interfaces that can be reviewed and refined quickly.
- Small, skilled team: With only three members, RAD's emphasis on small, focused teams with strong tool support fits naturally, avoiding the heavy process overhead of larger-scale models.
- Time-boxed delivery: The lab schedule requires a working MVP within a fixed, short timeframe — RAD's iterative build-review cycles are designed exactly for that kind of rapid, incremental delivery.
- Reusable components: Core modules (authentication, CRUD for rooms, booking logic) can be built using existing frameworks/libraries, reducing development time as RAD encourages component reuse.
- Frequent feedback loop: Each lab milestone effectively acts as a client review checkpoint, matching RAD's cycle of prototype → review → refine.

12. Comparison with Alternative Models

Why Waterfall is less suitable

Waterfall requires requirements to be completely finalized before design and implementation begin, with no iteration back to earlier phases. Since this project evolves across multiple labs and the client's expectations may be refined as the prototype is reviewed, Waterfall's rigid, sequential structure would make it difficult to incorporate feedback without restarting phases, increasing risk of a mismatch between the delivered system and actual needs.

Why Spiral is less suitable

Spiral is a risk-driven model best suited to large, complex, high-risk projects (e.g., safety-critical or large enterprise systems) where extensive risk analysis at every iteration is justified. For a semester-lab-scale MVP with a small team and low technical risk, Spiral's heavy risk-analysis overhead would consume time and effort disproportionate to the project's size and would slow down delivery of a working prototype.

Why Scrum is a reasonable but secondary alternative

Scrum could also work, since it is iterative and team-friendly. However, Scrum is process-heavy for a 3-person academic team — it assumes defined sprint ceremonies, a product backlog, and sprint reviews with a dedicated Product Owner. RAD achieves a similar iterative benefit with a lighter process, more suited to a small team working toward a fast, prototype-driven MVP within a single lab cycle.

Model	Fit for this project	Reason
RAD (Selected)	Best fit	Fast prototyping, small team, UI-focused, time-boxed delivery.
Waterfall	Poor fit	No room for iteration; requirements evolve across labs.

Spiral	Poor fit	Overhead of formal risk analysis not justified for project size.
Scrum	Possible, not chosen	Iterative but heavier ceremony/process than a 3-person team needs.